

hsh

Software Requirements Specification (SRS)

Natural-Language Enhanced Bash-Compatible Shell

Version 0.2 - Corrected architecture and safety contracts

Bash/C execution core + Rust intelligence, policy and provider layer

Status: Draft / Baseline for M0-M6

Date: 11 September 2026

Core principle: Bash executes deterministic shell semantics. AI proposes typed plans. hsh validates scope, privacy, policy, confirmation, and execution. Model output is never authority.

Document Control

Field	Value
Document version	0.2
Product	hsh
Architecture baseline	GNU Bash 5.3.x fork + Rust hsh-agent
Status	Draft / Corrected SRS
Primary platforms	Linux, macOS; WSL secondary
Previous version	0.1
Revision purpose	Close compatibility, context, execution schema, privacy, lifecycle, security, audit and milestone acceptance gaps

0.1 Revision summary

- Pins Bash compatibility baseline to GNU Bash 5.3.x and defines a compatibility contract and test gate.
- Distinguishes target natural-language UX from v0/M1 explicit `ai` / `?` routing.
- Defines typed conversational context, TTL, target snapshots, staleness checks and job boundaries.
- Defines confirmation UX and non-interactive behavior.
- Defines hsh-agent lifecycle and per-request latency requirements.
- Extends the execution schema to pipelines, redirections, declared glob expansion and privilege capabilities without raw eval.
- Adds host/environment risk profiles, canonical path scope and OS adapters.
- Makes prompt-injection resistance a normative security requirement.
- Defines provider data-egress classification and consent rules.
- Defines audit retention, integrity, redaction and access expectations.
- Adds GPL/legal boundary risk as an M0 release blocker.
- Maps acceptance criteria to milestones so M1 cannot claim M2/M3 functionality.

1. Purpose and Vision

hsh is a Bash-compatible command shell extended with a multilingual natural-language interaction layer. The product preserves ordinary Bash use while adding an explicit, inspectable path from natural language to typed execution plans.

The core product principle is: Bash remains the execution substrate; AI proposes or explains actions; hsh validates, authorizes, confirms when required, and executes those actions under deterministic local policy.

1.1 Target experience versus initial release

The following is the target UX, not the M1 input contract:

```
$ git status
$ liste meus arquivos em Downloads e me diga os que baixei na ultima semana
```

M1 requires explicit AI routing:

```
$ ai "liste os arquivos em Downloads que foram modificados na ultima semana"
$ ? por que o ultimo comando falhou?
```

Automatic natural-language routing is an opt-in M5 feature. Until M5, unprefixed input follows Bash command-resolution semantics and is never silently converted into an AI action.

1.2 Goals

- Preserve defined Bash 5.3.x compatibility for direct shell use.
- Provide multilingual natural-language interaction without creating a second command grammar.
- Use AI as planner/interpreter, never as an authorization or execution authority.
- Make destructive, privileged, networked and high-impact actions deterministic and inspectable.
- Support remote and local AI providers behind a stable provider contract.
- Support bounded conversational follow-ups without stale or cross-job target confusion.
- Keep direct Bash usable when AI is disabled, unavailable or crashes.

1.3 Non-goals for v0.x

- Full natural-language auto-routing before M5.
- Executing arbitrary model text with eval.
- Native Windows shell semantics before a dedicated Windows execution architecture.
- Autonomous background agents.
- Treating a process boundary as a legal conclusion about GPL derivative-work status.

2. Scope and Compatibility Contract

2.1 Bash baseline

COMP-001 The initial upstream compatibility baseline shall be GNU Bash 5.3.x. The exact upstream commit/tag and patch level shall be pinned in the build manifest and exposed by `hsh --version`.

COMP-002 For direct-command mode, hsh shall preserve Bash 5.3.x behavior for parsing, quoting, expansion, redirection, pipelines, job control, traps, functions, aliases, shell options, `set -o`, `shopt`, `PS1`, `PROMPT_COMMAND`, command history and programmable completion unless a documented hsh extension is explicitly enabled.

COMP-003 A change is compatibility-breaking if, for a direct command under equivalent environment and terminal conditions, observable parse result, exit status, stdout/stderr, side effects, signal behavior, job-control behavior, shell options, trap behavior or expansion result differs from the pinned upstream baseline without being listed as an intentional extension.

COMP-004 M0 shall pass the applicable upstream Bash test suite with no unexplained regressions. Any skipped, changed or failing test shall be documented with rationale and an hsh regression test.

COMP-005 hsh shall provide a compatibility-focused mode in which automatic AI routing and hsh syntax extensions are disabled.

2.2 Input ambiguity

COMP-006 Before M5, an unprefixed input such as `rm -rf /` is always treated as Bash input, not natural language. AI interpretation requires explicit `ai` or `?`.

COMP-007 In M5 automatic routing, syntactically valid Bash constructs and resolvable commands take precedence over AI routing unless the user explicitly invokes AI. Unknown-command fallback may offer AI interpretation but shall not execute the interpreted plan without showing that routing occurred.

COMP-008 Input that is valid Bash but resembles natural language shall never be reinterpreted after Bash has begun execution.

3. Users and Interaction Modes

Mode	AI allowed	Confirmation UI	Context	Primary purpose
------	------------	-----------------	---------	-----------------

Interactive direct Bash	No unless explicitly invoked	N/A	Bash session state only	Compatibility
Interactive `ai` / `?`	Yes	TTY confirmation contract	Bounded hsh context	M1-M4 AI interaction
Interactive auto-routing	Yes, opt-in M5	TTY confirmation contract	Bounded hsh context	Target hybrid UX
`hsh -c` / script	Disabled by default	No interactive prompt	No conversational context by default	Deterministic automation
Explicit non-interactive AI API	Only with explicit flag/config	No implicit approval; policy token required for mutations	Request-scoped only	Future automation

UX-001 Pressing Enter at a confirmation prompt shall mean cancel/deny, never approve.

UX-002 Confirmation syntax shall default to `[y/N]`; acceptance requires an explicit affirmative token. Locale-specific affirmative tokens may be supported but shall not make empty input affirmative.

UX-003 A confirmation request shall expire after a configurable timeout; default 60 seconds. Expiration denies execution.

UX-004 The preview shall show step type, executable, arguments, working directory, redirections, declared target set or target pattern, capability set and risk reason. Destructive multi-target operations shall show a bounded sample plus total count when discoverable.

UX-005 Policy may require confirmation per plan or per destructive step. The default for M2 is per plan unless a later step expands the target set, in which case re-confirmation is required.

UX-006 AI-mediated mutation shall be disabled in non-interactive mode by default. Enabling it requires an explicit policy configuration and a non-interactive authorization mechanism; stdin must not be treated as an implicit confirmation channel.

4. System Architecture

4.1 Logical flow

```

User input
|
+---> Direct Bash -----> Bash/C core
|
+---> Explicit AI / auto-route (M5)
      -> hsh bridge
      -> hsh-agent (Rust)
      -> Provider adapter
      -> Typed Plan
      -> Schema validator
      -> Privacy / data-egress gate
      -> Policy + scope evaluator
      -> Confirmation gate
      -> Execution broker
      -> Bash/system process primitives
      -> Result envelope
      -> Optional summarizer

```

4.2 Agent lifecycle

ARCH-001 M1-M4 shall use one hsh-agent child process per interactive hsh session, started lazily on first AI invocation. A machine-wide daemon is out of scope unless introduced by a later ADR.

ARCH-002 The Bash/C parent owns the agent lifecycle. The parent shall terminate the agent on shell exit and shall detect unexpected agent termination.

ARCH-003 If hsh-agent crashes or is unavailable, direct Bash continues. No pending AI plan shall execute after loss of the policy/agent channel.

ARCH-004 OS child processes already started by the execution broker are owned by an execution record. Agent failure shall not cause Bash job-control corruption; foreground AI-mediated execution shall be cancelled or reaped according to the recorded execution policy, while explicitly backgrounded operations require a later dedicated design and are out of scope through M3.

ARCH-005 The C<->Rust boundary shall use a versioned, length-delimited JSON message protocol over pipes or local sockets. Protocol mismatch shall fail closed for AI execution.

ARCH-006 A machine-wide daemon, persistent cross-shell memory and autonomous background tasks are not part of M1-M4.

5. Performance and Availability Requirements

NFR-001 Direct Bash startup and direct-command latency shall remain within 10% of the pinned upstream Bash baseline on the same host for the release benchmark suite, excluding first-run packaging effects.

NFR-002 With an already-running hsh-agent and a local mock provider, bridge + validation overhead excluding model inference and OS command runtime shall have p95 <= 100 ms on the reference development host.

NFR-003 The first AI invocation may pay lazy agent startup cost; local agent startup p95 target is <= 300 ms on the reference host, excluding provider/network latency.

NFR-004 Provider requests shall have configurable connect and total timeouts. Default total provider timeout shall be 30 seconds for interactive planning unless the provider adapter defines a stricter value.

NFR-005 Ctrl-C during provider planning shall cancel the provider request and return control to the interactive shell without terminating hsh.

NFR-006 Provider outage, malformed output, timeout or agent failure shall never prevent direct Bash use in the same session where feasible.

6. Functional Requirements

FR-001 hsh shall execute direct Bash input according to the compatibility contract in Section 2.

FR-002 M1 shall provide explicit AI invocation through `ai` and optionally `?`.

FR-003 Automatic natural-language routing is an opt-in M5 capability and is not an M1 acceptance requirement.

FR-004 The AI path shall support multilingual requests without per-language command grammars.

FR-005 Every AI plan shall conform to a versioned typed schema before policy evaluation.

FR-006 Plan steps shall represent executable programs and argument arrays separately. Raw shell text is not an executable plan type through M3.

FR-007 Every plan shall declare requested capabilities; hsh independently derives effective capabilities and risk from the normalized plan.

FR-008 No model-provided risk label or instruction can lower locally derived risk.

FR-009 Every mutating, destructive, privileged or scope-expanding action shall pass policy and the confirmation contract.

FR-010 Policy denial shall produce no requested side effect.

FR-011 Read-only actions may execute without confirmation only if local policy, path scope and privacy policy permit.

FR-012 Execution results shall include per-step status, stdout/stderr metadata, exit status, duration and resolved target metadata where applicable.

FR-013 Natural-language summaries shall be optional and shall be subject to provider data-egress policy.

FR-014 The user shall be able to inspect the complete normalized plan before approval.

FR-015 Provider adapters shall be replaceable without modifying the Bash core.

FR-016 hsh shall support offline/no-AI mode with direct Bash fully usable.

FR-017 hsh shall expose hsh version, Bash baseline and protocol schema versions.

FR-018 An in-progress AI request shall be cancellable independently of the shell session.

FR-019 M2 shall implement write/destructive capability enforcement before destructive acceptance criteria are enabled.

FR-020 M3 shall implement conversational reference resolution before contextual destructive follow-ups are enabled.

7. Execution Plan Protocol

7.1 Required step types

The plan protocol models shell composition explicitly. It does not encode a pipeline as a single free-form shell string.

Step type	Purpose	Key fields	M1-M3 policy
process	Execute one program	program, args, cwd, env_delta, timeout	M1
pipeline	Connect typed process stages	stages[], pipefail	M1 read-only; mutation M2
redirection	Attach file/std streams	stream, mode, path, append	Read M1; write M2
glob	Declared path expansion	pattern, base_dir, match_policy	Read M1; mutation M2
builtin	Approved hsh/Bash builtin subset	name, args	Allowlist only
privilege	Request OS privilege boundary	mechanism, nested_step	M2+; explicit capability
shell_script	Raw shell program text	N/A	Forbidden through M3; future ADR required

7.2 Example typed pipeline

```
{
  "schema_version": "1.1",
  "request_id": "...",
  "steps": [{
    "type": "pipeline",
    "pipefail": true,
    "stages": [
      {"type": "process", "program": "find", "args": ["/Users/user/Downloads", "-type", "f", "-mtime", "-7"]},
      {"type": "process", "program": "sort", "args": ["-r"]}
    ]
  }]
}
```

7.3 Globbs, redirections and sudo

PLAN-001 Glob expansion shall be represented explicitly. The executor may perform expansion only from a declared pattern and base directory; resulting paths become part of the resolved target set used for policy evaluation.

PLAN-002 Output redirection that creates, truncates or appends a file is a filesystem.write capability even when every process in the pipeline is otherwise read-only.

PLAN-003 `sudo` shall not be treated as an ordinary benign program. It requires `privilege.escalate` and a privilege step or equivalent policy-recognized representation.

PLAN-004 Command substitution, process substitution, here-documents and arbitrary compound Bash syntax are not representable as model-generated executable plans through M3 unless a dedicated typed step is defined and security-reviewed.

PLAN-005 The executor shall never reconstruct a shell command string merely to invoke `bash -c` for typed process/pipeline steps.

8. Conversational Context Contract

8.1 Context objects

Context stores typed references, not only raw stdout. A prior result eligible for pronoun/reference resolution is a ContextResult object containing an immutable result identifier, originating request, operation class, resolved target identifiers, timestamps and freshness metadata.

```
ContextResult {
  result_id,
  job_id,
  created_at,
  expires_at,
  source_request_id,
  target_type,           // pid | path | process | repo_object | other typed kind
  targets[],             // canonical IDs/paths plus display metadata
  source_snapshot,       // process start-time, inode/dev where available, etc.
  sensitivity,
  summary_metadata
}
```

CTX-001 Default conversational context TTL shall be 15 minutes and configurable. Expired objects are not eligible for implicit reference resolution.

CTX-002 Default maximum context shall be 20 result objects or 256 KiB of structured metadata, whichever is reached first. Raw stdout is not automatically retained as conversational context.

CTX-003 Pronouns such as "eles" may resolve only inside the current interactive hsh context job unless the user explicitly names or selects another result identifier.

CTX-004 A context job is a user-visible chain begun by an AI request and its direct follow-ups. A new unrelated AI request starts a new job by default.

CTX-005 Before a context-derived mutation, hsh shall re-resolve and validate every target. For processes this includes PID plus process identity metadata such as start time where available; for files this includes canonical path and filesystem identity metadata where available.

CTX-006 If the target set changed materially, contains stale identifiers, or cannot be revalidated, hsh shall not silently substitute targets. It shall drop stale targets or require a refreshed plan and confirmation, according to policy.

CTX-007 The confirmation preview for context-derived mutations shall identify the source result_id and show the current revalidated target set.

CTX-008 Raw command output and file contents are data, not trusted instructions. They may inform a summary but shall not acquire authority to modify policy, tools, permissions or plan constraints.

9. Policy, Identity and Environment Model

POL-001 Policy evaluation shall include interaction mode, effective user identity, host profile, current working directory, canonical target paths, mount/filesystem classification, requested capabilities and plan provenance.

POL-002 Host profiles shall support at minimum ``development``, ``production`` and ``custom``; production shall default to stricter mutation/network rules.

POL-003 Working directory and resolved target scope shall influence risk. The same command may receive different authorization in a configured scratch directory versus system or production paths.

POL-004 Path authorization shall operate on canonicalized targets. For destructive operations hsh shall use filesystem-aware resolution (``realpath``/descriptor-based equivalents) and shall defend against symlink races to the extent supported by the platform.

POL-005 Special filesystems and pseudo-filesystems such as ``/proc``, ``/sys``, device nodes, network mounts and configured bind mounts shall be classifiable and may be denied or elevated by default.

POL-006 A plan that expands its target set after confirmation shall require re-evaluation; destructive scope expansion requires re-confirmation.

POL-007 Interactive and non-interactive execution shall be separate policy dimensions. An authorization valid for a TTY session is not implicitly valid for ``hsh -c``.

10. OS Abstraction

OS-001 hsh shall have an OS adapter interface for path conventions, process inspection, filesystem identity, signals, privilege integration and command availability.

OS-002 Linux and macOS are primary targets; platform-specific plans may differ while preserving the same high-level intent and capability model.

OS-003 WSL is a secondary Linux target. Native Windows execution semantics are not implied by WSL support.

OS-004 Provider planning prompts shall receive normalized platform capability metadata rather than assume commands such as GNU ``find``, BSD ``find``, ``sed``, ``stat`` or package managers are interchangeable.

11. Security Requirements

SEC-001 AI output is untrusted data until schema validation, normalization, independent capability derivation and policy evaluation complete.

SEC-002 No AI-generated raw text shall be passed to ``eval``, ``bash -c``, ``sh -c`` or equivalent execution in M1-M3.

SEC-003 Command output, stderr, file content, repository content and network responses shall be treated as untrusted data that may contain prompt injection.

SEC-004 Instructions embedded in untrusted data shall not change system/developer policy, enable capabilities, bypass confirmation, select a different provider, disable logging or alter privacy settings.

SEC-005 The provider prompt contract shall clearly separate user intent, trusted policy metadata and untrusted observations using typed fields or equivalent structural boundaries.

SEC-006 The local policy engine shall ignore model requests to weaken or disable policy.

SEC-007 A model cannot approve its own plan, confirmation or privilege escalation.

SEC-008 Destructive path operations shall protect against path traversal, symlink substitution and time-of-check/time-of-use risk using platform-appropriate primitives.

SEC-009 Secrets shall not be included in provider requests or audit logs by default.

SEC-010 Security tests shall include adversarial stdout/file content attempting to instruct the model or hsh to bypass policy.

SEC-011 Malformed, unknown or future schema fields that affect execution shall fail closed unless explicitly supported by the current protocol version.

12. Privacy and Provider Data-Egress Policy

12.1 Data classes

Class	Examples	Default remote-provider rule
Metadata	exit code, file count, sizes, timestamps, command class	Allowed if needed
User request	natural-language text, selected cwd label	Allowed after secret redaction
Paths	absolute/relative filenames and directories	Minimize; redact home/user segments where feasible
Command output	stdout/stderr body	Denied by default; explicit task need or user opt-in
File content	source code, documents, config	Denied by default; explicit content task + policy
Secrets	tokens, passwords, private keys, secret env vars	Never send by default; explicit unsafe override not provided in v0.x

PRV-001 Every provider request shall be assembled by a data-egress gate from explicitly classified fields. Provider adapters shall not receive unrestricted shell state.

PRV-002 Environment variables are excluded by default. An allowlist may expose specific non-secret variables when required for planning.

PRV-003 FR-style failure explanation shall first use local metadata. Sending stderr/stdout body to a remote provider requires either explicit user invocation that clearly requests analysis of that output or a configured consent policy.

PRV-004 Natural-language result summaries may be generated locally from structured results. Remote summarization of output body follows the same content-egress rule as PRV-003.

PRV-005 Local providers may use a separate privacy profile, but they remain subject to prompt-injection and policy rules.

PRV-006 The preview/debug UI shall allow users to inspect which data classes are being sent to the provider without exposing redacted secret values.

13. Audit and Observability

AUD-001 Audit records shall include request_id, timestamp, interaction mode, provider identity, schema version, normalized plan hash, derived capabilities, policy decision, confirmation outcome, execution status and redacted target metadata.

AUD-002 The original natural-language request shall not be logged verbatim by default. A redacted form or stable hash may be stored; full request logging is opt-in.

AUD-003 Default local retention shall be 30 days or 50 MiB, whichever limit is reached first, with rotation. Configuration may reduce or disable non-security diagnostic retention.

AUD-004 Audit files shall use owner-only permissions by default. Append-only chaining with record hashes shall be supported for tamper evidence; hsh shall not claim protection against a fully privileged local root administrator.

AUD-005 Audit access shall be limited to the owning user by default. Administrative aggregation is outside M1-M3 and requires separate policy/design.

AUD-006 Redaction shall occur before serialization to persistent audit storage.

14. Error Handling

Failure	Required behavior	Side-effect rule
Provider unavailable	Report error; direct Bash remains usable	No plan execution
Invalid/malformed plan	Reject and identify schema issue	No execution

Policy denial	Report denied capability/scope	No requested side effect
Stale context target	Re-resolve, drop stale or request refresh	No substitution to new target
Confirmation timeout/cancel	Return denied/cancelled	No execution
Agent crash	Cancel pending AI plan; preserve shell	No pending AI execution
Command failure	Capture exit status/stderr metadata; optional explanation	Only already-started step effects remain
Privacy gate denial	Explain required data class	No provider transmission of denied field

15. Licensing and Distribution Risk

hsh forks GNU Bash and therefore inherits GPL obligations for Bash-derived portions. The engineering architecture shall not treat process separation as a guarantee that a Rust component is legally independent.

LIC-001 M0 shall record the exact Bash license/copyright provenance and third-party dependency licenses.

LIC-002 Before public distribution, legal review shall classify the Bash-derived executable, hsh bridge, Rust agent, packaging and provider modules under the actual distribution/linkage/IPC design.

LIC-003 No architecture decision shall claim that a separate process or package automatically removes GPL obligations; this remains a legal risk until reviewed.

LIC-004 Upstream Bash security fixes shall be tracked and merged according to a documented maintenance policy. This section defines engineering release gates and is not legal advice.

16. Repository Structure

```

hsh/
  bash/           # pinned Bash 5.3.x-derived C core
  bridge/         # C-side hsh integration
  rust/
    hsh-agent/    # session-scoped orchestration
    hsh-protocol/ # versioned typed plan/result schema
    hsh-policy/   # deterministic authorization/risk
    hsh-context/  # typed bounded context
    hsh-privacy/  # provider data-egress classification
    hsh-os/       # Linux/macOS/WSL adapters
    hsh-provider/ # provider traits
    providers/
      openai/
      ollama/
  config/
  docs/
  tests/
    bash-compat/
    protocol/
    policy/
    context/
    privacy/
    security/
    integration/
  packaging/

```

17. Verification and Test Strategy

17.1 Compatibility

- Run pinned upstream Bash 5.3.x tests and record all deltas.

- Add dedicated tests for PS1, PROMPT_COMMAND, programmable completion, set -o/shopt, traps, job control and Ctrl-C while hsh-agent exists.
- Compare direct-command parse/exit/output/side-effect behavior with pinned Bash under controlled fixtures.

17.2 Protocol and execution

- Typed process, pipeline, redirection and glob golden tests.
- Ensure shell metacharacters stay literal arguments unless represented structurally.
- Ensure no typed plan path invokes raw eval or `bash -c`.
- Test scope expansion between preview and execution.

17.3 Context

- TTL expiration and max-size eviction.
- PID reuse / process identity mismatch.
- Deleted/recreated file path identity changes.
- No cross-job implicit pronoun resolution.
- Fresh risk evaluation for every context-derived mutation.

17.4 Security/privacy

- Prompt-injection strings in stdout, stderr, file content and network responses.
- Model attempts to disable policy, change privacy or self-confirm.
- Secret/env redaction before provider and audit serialization.
- Remote provider blocked from output/file bodies unless task policy authorizes it.
- Symlink/path traversal and TOCTOU-sensitive destructive fixtures.

18. Delivery Roadmap and Milestone Gates

Milestone	Scope	Required ACs	Explicitly not complete yet
M0 - Fork baseline	Pinned Bash 5.3.x, rename/build, compatibility harness, licensing provenance	AC-001, AC-002, AC-003	AI execution, mutation, context
M1 - Read-only AI MVP	`ai`/`, session agent, one provider, typed process/pipeline/read redirection/glob, privacy gate	AC-101..AC-109	Writes/destructive actions; contextual destructive follow-up
M2 - Mutations + policy	Write/delete/process.kill/privilege capabilities, confirmation UX, path/host profiles	AC-201..AC-208	Contextual pronoun mutation
M3 - Bounded context	Typed ContextResult, TTL, job boundaries, stale-target revalidation	AC-301..AC-307	Automatic NL routing
M4 - Multi-provider	Remote + local provider switching, provider failure isolation	AC-401..AC-404	Automatic NL routing
M5 - Smart routing	Opt-in automatic NL detection + ambiguity contract	AC-501..AC-505	Native Windows
M6 - Hardened beta	Packaging, update path, audit hardening, docs, release gates	AC-601..AC-605	Unreviewed raw shell-script plan step

19. Acceptance Criteria by Milestone

19.1 M0

AC-001 `hsh` launches interactively and direct commands match the pinned Bash 5.3.x compatibility suite within documented deltas.

AC-002 `hsh --version` reports hsh version and exact upstream Bash baseline.

AC-003 Licensing provenance and the unresolved combined-work/process-boundary legal risk are recorded as release artifacts.

19.2 M1 - MVP

AC-101 `ai liste os arquivos desta pasta` produces a schema-valid read-only typed plan and executes without raw eval.

AC-102 A read-only pipeline is represented as structured stages, not a shell string.

AC-103 A read-only glob is expanded by the executor and its resolved set is policy-visible.

AC-104 An attempted write/delete/kill/privilege plan is denied as unsupported in M1 rather than executed.

AC-105 Provider failure, timeout or agent crash leaves direct Bash usable.

AC-106 Ctrl-C cancels planning and returns control to hsh.

AC-107 The user can inspect the normalized plan before execution.

AC-108 Remote provider requests exclude env vars, secrets, file bodies and command-output bodies by default.

AC-109 Bridge/validation overhead meets NFR-002 with the local mock provider.

19.3 M2

AC-201 Destructive requests require `[y/N]`; Enter, timeout and negative answers deny execution.

AC-202 Confirmation shows executable, args, cwd, redirections, capabilities and target scope.

AC-203 Policy denial produces no requested side effect.

AC-204 Production host profile is stricter than development defaults.

AC-205 Symlink/path-scope destructive tests pass.

AC-206 Output redirection to a file is classified as filesystem.write.

AC-207 `sudo`/privilege use requires explicit privilege capability and OS authentication.

AC-208 Non-interactive AI mutation remains denied unless explicitly configured with a non-interactive authorization mechanism.

19.4 M3

AC-301 A follow-up such as `mate todos eles` resolves only to a current ContextResult in the same job.

AC-302 Context expires according to TTL and size bounds.

AC-303 PID reuse or identity mismatch prevents accidental kill of a new process.

AC-304 Deleted/recreated file targets are detected as stale or changed before mutation.

AC-305 Context-derived mutations undergo fresh policy evaluation and confirmation.

AC-306 Cross-job pronouns do not implicitly target older results.

AC-307 Untrusted prior stdout cannot alter policy, privacy or confirmation behavior.

19.5 M4-M6

AC-401 At least one remote and one local provider implement the same provider contract.

AC-402 Switching providers does not change execution schema or policy semantics.

AC-403 Provider-specific failures do not crash the shell.

AC-404 Provider data-egress inspection works consistently across adapters.

AC-501 Automatic NL routing is opt-in and disabled by default on upgrade.

AC-502 Valid Bash syntax takes precedence over auto-routing unless AI is explicitly invoked.

AC-503 Unknown-command AI fallback clearly indicates rerouting before planning.

AC-504 Ambiguous high-risk intent never auto-executes.

AC-505 Compatibility mode disables automatic routing.

AC-601 Packaging preserves required license notices and provenance.

AC-602 Audit retention/redaction/integrity tests pass.

AC-603 Security regression suite includes prompt injection, scope expansion and stale context.

AC-604 Linux and macOS adapters pass release fixtures; WSL status is documented.

AC-605 No unreviewed raw shell-script execution path exists in release builds.

20. Architecture Decisions Baseline

ID	Decision	Choice
ADR-001	Product name	hsh
ADR-002	Shell foundation	Fork GNU Bash 5.3.x
ADR-003	Bash core language	C
ADR-004	Intelligence/policy layer	Rust
ADR-005	Initial C/Rust integration	Session child process + versioned JSON
ADR-006	Execution model	Typed plan; no raw eval
ADR-007	Configuration	TOML
ADR-008	AI routing rollout	Explicit first; auto-routing M5
ADR-009	Context model	Typed result references with TTL/job scope
ADR-010	Provider privacy	Explicit data-egress gate
ADR-011	OS portability	OS adapter layer
ADR-012	Raw shell-script plan	Forbidden through M3; later ADR/security review required

21. Risks and Open Decisions

Risk	Impact	Current mitigation / gate
Bash fork divergence	Compatibility/security regressions	Pinned baseline, upstream test suite, documented deltas
Ambiguous NL routing	Unexpected actions	Explicit routing until M5; Bash precedence
Stale conversational targets	Wrong destructive action	Typed context, TTL, identity revalidation, fresh confirmation
Prompt injection from local data	Policy bypass/data leak	Data/instruction separation + local deterministic policy
Provider data leakage	Secrets/source exposure	Data classification + egress gate + redaction
C/Rust lifecycle complexity	Zombie jobs / broken terminal state	Session-owned agent; direct Bash fallback; no background AI jobs through M3

GPL boundary assumption	Distribution/legal exposure	M0 provenance + pre-distribution legal review
OS command differences	Incorrect plan semantics	OS adapters and capability metadata

22. Glossary

Term	Definition
Direct Bash	Input executed under Bash semantics without AI planning.
AI-mediated command	An operation whose plan is produced or interpreted through an AI provider.
Execution plan	Versioned typed representation of proposed actions.
Capability	Policy permission category such as filesystem.read, filesystem.write, process.kill or privilege.escalate.
ContextResult	Typed, bounded reference to a prior inspectable result and its target identities.
Context job	User-visible chain of one AI request and its direct follow-ups.
Data-egress gate	Local component deciding which classified data fields may be sent to an AI provider.
Provider	Remote or local AI backend implementing the hsh provider interface.
Host profile	Policy profile such as development or production.
Compatibility mode	Mode disabling hsh auto-routing/extensions that can create ambiguity with Bash.

End of SRS - hsh v0.2

hsh v0.2.1 - Normative PDF Patch

Applies to: hsh_SRS.pdf | Source SHA-256: e3e0c642bcd42636cc363b998e993e64ffe9488ca45a3f9a4c9541d5df9e48df

PATCH PRECEDENCE

This appendix is normative. Where it conflicts with text, examples, tables, milestone labels, repository layouts, acceptance-criterion identifiers, or traceability statements in the v0.2 body, this appendix prevails. Unchanged v0.2 requirements remain in force.

AB-001 - Canonical milestone and acceptance-criterion identifiers

The only milestone identifiers are **M0 through M6**. Acceptance criteria use the global form **M<n>-AC-<nnn>**. Legacy IDs are searchable aliases only and MUST NOT be used in new feature plans, pull requests, ADRs, test names, or release gates.

Legacy source	Canonical mapping rule
SRS AC-001..AC-003	AC-00x -> M0-AC-00x
SRS AC-101..AC-109	AC-1xy -> M1-AC-0xy
SRS AC-201..AC-208	AC-2xy -> M2-AC-0xy
SRS AC-301..AC-307	AC-3xy -> M3-AC-0xy
SRS AC-401..AC-404	AC-4xy -> M4-AC-0xy
SRS AC-501..AC-505	AC-5xy -> M5-AC-0xy
SRS AC-601..AC-605	AC-6xy -> M6-AC-0xy
Roadmap Mx-AC-01..99	Mx-AC-yy -> Mx-AC-1yy (roadmap-specific gate range)
Roadmap M0-AC-06	Alias of M0-AC-003; checklist detail remains normative.
UML component exits	Reference canonical Mx-AC sets; no parallel AC numbering.

Examples: SRS AC-104 becomes **M1-AC-004**; Roadmap M1-AC-07 becomes **M1-AC-107**. They are distinct criteria and no longer collide.

AB-003 - Protocol and ExecutionPlan version

The frozen M0 baseline is **IPC protocol_version = "1.0"** and **ExecutionPlan schema_version = "1.0"**. The SRS example containing **"schema_version": "1.1"** is corrected to **"1.0"**. A later 1.1 bump requires an explicit compatibility rule and ADR.

AB-004 - AuthorizationArtifact exists in M1

Every AI-generated plan that reaches the canonical Bash executor MUST carry a locally issued **AuthorizationArtifact**, including read-only M1 plans. M1 read-only authorization does not require user confirmation when policy allows it; M2 extends the same artifact with confirmation binding for state-changing operations.

The artifact binds at minimum **plan_hash**, **session_id**, **restart_epoch**, **immutable policy/config snapshot hashes**, **uid/gid/cwd**, **effective capabilities**, **issued_at** and **expires_at**. The Bash Compatibility Core SHALL reject AI-plan execution without a valid artifact.

hsh v0.2.1 - Normative PDF Patch

Applies to: hsh_SRS.pdf

AB-002 - Canonical repository layout

The following tree is the single repository baseline. Any alternate tree in the v0.2 body is superseded. No Rust crate may own direct OS process execution for AI-generated plans.

```
hsh/
  bash/                # pinned GNU Bash 5.3.x-derived C core
  bridge/              # C <-> agent IPC integration
  builtins/            # hsh-specific Bash/C builtins
  schemas/
    protocol/
    execution-plan/
  rust/
    hsh-agent/         # per-interactive-session orchestrator
    crates/
      hsh-protocol/
      hsh-intelligence/
      hsh-policy/
      hsh-context/
      hsh-egress/
      hsh-authorization/
      hsh-audit/
      hsh-os/           # metadata/canonicalization only
      hsh-provider/
    providers/
      openai/
      ollama/
  tests/
    compatibility/
    protocol/
    policy/
    adversarial/
    integration/
```

AB-005 - Platform status: product target vs. implementation gate

Product targets: Linux and macOS. **M1 blocking platform:** exactly one of Linux or macOS, selected during M0. The other is smoke/non-blocking through M5 and becomes blocking by M6. WSL remains secondary/documented unless promoted by ADR.

AB-006 - M1 explicit AI surface and Data-Egress Gate

The mandatory M1 surface is **ai "..."**. The **?** shorthand is optional syntax sugar and **MUST NOT** be an M1 release gate. Data-Egress Gate is P0/M1 and runs before every remote-provider transmission.

M1 data class	Remote-provider default
User NL request	Allowed after local redaction/classification
Typed OS metadata required to plan	Allowlisted only
Env vars / credentials / secrets	Denied
File/repository bodies	Denied
stdout / stderr bodies	Denied
Exit status, bounded paths/sizes/counts	Local presentation; egress only if later classified

M1 natural-language-like result presentation MUST be local/deterministic or metadata-only. It SHALL NOT perform a second provider call containing raw stdout/stderr or file bodies.

AB-007 - Schema-known step types vs. milestone authorization

ExecutionPlan v1 MAY define FileOperationStep, ProcessOperationStep, NetworkStep and PrivilegeStep from M0. Schema recognition does not imply authorization. In M1, any plan requiring capabilities outside the read-only M1 set **MUST** be rejected as **unsupported/denied before AuthorizationArtifact issuance**. Write, delete, process control/kill, network side effects and privilege escalation remain unavailable in M1.

Capabilities are always recomputed locally from typed steps, resolved targets, cwd, env_delta, redirections, host profile and filesystem metadata. Provider risk/capability fields are non-authoritative hints.

hsh v0.2.1 - Normative PDF Patch

Applies to: hsh_SRS.pdf

AB-008 - UML M6 component exit is normative

The UML component-design traceability section is extended through **M6**. M6 does not add an alternate executor or trust path. Component exit requires:

- compatibility regression against the pinned Bash baseline and documented intentional deviations;
- Linux and macOS release fixtures blocking by 1.0; WSL status documented;
- fuzzing/adversarial coverage for JSON/IPC, typed plan decoding, policy inputs and AuthorizationArtifact verification;
- agent crash/restart isolation that cannot corrupt Bash job-control state;
- audit retention, redaction and integrity controls finalized;
- packaging/license/provenance checks completed; stable v1 protocol/provider contracts frozen;
- no unreviewed raw shell-script execution path in release builds.

AB-009 - Canonical sequence

M0 Foundation -> M1 Explicit AI/read-only + Policy v1 -> M2 Controlled Mutation -> M3 Bounded Context -> M4 Provider/Config Maturity -> M5 Opt-in Automatic NL Routing -> M6 Hardening/1.0.

No milestone may enable a capability assigned to a later milestone merely because its step type already exists in the schema.

AB-010 - Canonical executor and trust path

The only OS process executor for AI-generated plans is the Bash Compatibility Core in C. Rust may plan, normalize, recompute capabilities, evaluate policy, build context, apply data-egress policy, issue authorization metadata and coordinate IPC; Rust SHALL NOT call exec/fork/spawn as an alternate AI-plan execution path.

```
Frontend C / TTY
-> hsh-agent (plan + policy)
-> AuthorizationArtifact
-> Frontend C / confirmation when required
-> Bash Compatibility Core
-> OS fork/exec/job-control/redirection
-> bounded local result
-> optional typed ContextResult (M3+)
```

AB-011 - Roadmap token estimates

Roadmap token ranges are planning heuristics for assisted engineering work only. They are not Codex/OpenAI credits, API runtime budget, a contractual delivery estimate, or an acceptance criterion. Measurement reports SHOULD record model/configuration, input/output mix, whether upstream Bash source was inspected or modified, and excluded generated artifacts.

Migration note

Future v0.3+ editions should fold these rules into the body and remove legacy aliases. Until then, implementation artifacts SHALL cite the canonical IDs and baseline in this appendix.